

LISTENING EDITION

ChatGPT Work and Codex Site Tools Today

A speech-friendly guide to the page-scoped WordPress WebMCP adapter in the ChatGPT desktop app's built-in browser.

Verified 3 September 2026. This edition uses one-column reading order, short sections, and plain language instead of dense code listings.

The short answer

A WordPress page can register named JavaScript tools on document `.modelContext`. ChatGPT Work or Codex can discover them, send schema-shaped arguments, and receive a structured result from the live page.

The user and agent share the current page, signed-in session, visible controls, editor selection, and unsaved Gutenberg state. This is page collaboration, not an independent backend MCP connection.

The boundary

The adapter projects only WordPress client Abilities. It rejects REST-backed Ability records and never requests the server Ability catalog. WordPress and third-party providers own page logic; the adapter owns WebMCP projection, lifecycle, confirmation, activity, and redacted observability.

Where tools appear

Each document receives only the providers that apply to it. Counts are stable acceptance contracts, not settings.

Page	Tools	What is added
Anonymous frontend	2	Page context and rendered site destinations
Authenticated frontend	3	Frontend tools plus rendered admin-toolbar destinations
Generic wp-admin	2	Page context and rendered admin-menu destinations
General Settings	3	One safe form-staging Ability
Compatible block editor	17	Two admin reads plus 15 editor tools
Authentication screen	0	No adapter assets or activity UI

Navigation

Destination tools read navigation that WordPress rendered for the current user. They return safe same-origin URLs. The browser opens a result normally, then the agent rediscovers the new document's inventory.

Names and lifecycle

A WordPress Ability such as `webmcp/editor-context` becomes `webmcp.editor-context`. The slash-to-dot mapping keeps segment boundaries. The adapter waits for registration, removes tools when their Abilities disappear, rejects stale handles, and handles late registration without duplicates.

Working in Gutenberg

Every compatible post, page, custom-post-type, and Site Editor screen receives the same 15 editor tools. WordPress decides block-editor compatibility through the current screen; the adapter keeps no post-type list.

Six reads

- Editor context and current selection.
- The live recursive block tree.
- Installed block types and their contracts.
- Theme design tokens.
- Available patterns.
- Templates and template parts.

Eight reversible writes

- Insert, update, move, replace, or remove blocks.
- Insert a pattern.
- Edit supported post attributes without changing status.
- Undo or redo staged editor changes.

One consequential save

`webmcp.save-post` persists staged state and owns explicit publishing. It always opens the trusted in-page confirmation. A no-change call reports that nothing was saved.

Block operations use one recursive shape: `{ name, attributes, innerBlocks }`. Callbacks re-read transient block IDs and live editor state before changing anything, then verify the observed result.

Staging General Settings

`wordpress.settings.stage-general-form` appears only on General Settings. It accepts a closed set of supported fields, validates the whole request against live controls, and changes only the fields provided.

- Uses native value setters and normal input and change events.
- Supports preset and custom date and time formats.
- Highlights changed controls and adds a review notice.
- Never submits the form and sends no persistence request.
- Reloading discards the staged values.

Administration Email is sensitive. The address never appears in the result, review feedback, activity request, stored event, or exporter hook. Only the field name and the WordPress confirmation warning are retained.

Risk and confirmation

Readonly Abilities derive read risk. Every mutation declares reversible, persistent, consequential, or privileged risk. Missing or invalid mutation metadata fails closed.

Consequential and privileged calls show provider, action, page context, risk, and a bounded redacted summary. Approval requires a trusted click. Decline, Escape, 60-second expiry, forwarded cancellation, and the final context check all stop execution.

The built-in browser also performs its own invocation review. That product layer does not replace WordPress authorization or the provider's validation.

Activity and observability

Eligible pages start with one minimized 48-pixel activity button. A badge marks new activity. The detail region opens only when requested and shows running, ran, failed, declined, expired, cancelled, and stale outcomes.

- Keyboard access with Enter, Space, and Escape.
- Focus restoration and polite screen-reader announcements.
- Responsive layout and isolated styles.
- Safe text rendering and same-origin activity links.
- Open or minimized state remembered only for the current tab.

Backend events

After an invocation settles, the adapter sends one fire-and-forget event. Observability failure never changes the tool result.

The server accepts only bounded identifiers and classifications, normalized page context and path, hashed anonymous identity or WordPress user ID, outcome, duration, confirmation result, safe error code, and an optional allowlisted summary. It excludes raw inputs, outputs, page content, arbitrary errors, IP addresses, and login-session tokens.

Anonymous ingestion uses short-lived signed context tokens, a 4 KB payload limit, and hashed fixed-window rate limits. Default retention is seven days and 10,000 rows. Administrators review normalized and labeled legacy rows under Tools, Site tools activity.

Client behavior and verification

ChatGPT Work and Codex currently discover imperative tools registered by the top-level document. They do not discover declarative form tools or iframe registrations. A navigation or reload replaces the document and requires fresh discovery.

Same-document refresh timing and callback cancellation forwarding are not guaranteed. Providers keep live guards; confirmation keeps its expiry bound. Current system Chrome is a deterministic page-side harness, while Codex in the built-in browser remains the product acceptance target.

Acceptance coverage

- Dashboard, General Settings, post, page, compatible custom-post-type, and Site Editor pages.
- Site Editor inventory across a visible in-shell Pages route.
- Authenticated and anonymous home and singular pages.
- Both third-party provider fixture pages, removal, stale calls, restoration, and reversal.
- Login, password-reset, and registration exclusion.
- Rendered navigation, staging without persistence, editor undo, trusted decline and approval, cancellation, and expiry.
- Authenticated and anonymous observability, exporter hooks, rate limits, retention, legacy migration, redaction, and uninstall-only cleanup.

Sources

OpenAI Site tools: <https://learn.chatgpt.com/docs/webmcp>

Chrome imperative WebMCP API: <https://developer.chrome.com/docs/ai/webmcp/imperative-api>

WebMCP specification: <https://webmachinelearning.github.io/webmcp/>

Project: <https://github.com/galatanovidiu/webmcp-adapter>